

MC714 - Sistemas Distribuídos

Relatório técnico e de auditoria do projeto

Implementação de sistemas distribuídos com HTTP/JSON entre
contêineres

Autores: José Eduardo Santos Rabelo (RA 260551)
Raphael Salles Vitor de Souza (RA 223641)
Professor: Luiz Fernando Bittencourt
Unidade: IC - Instituto de Computação da Unicamp, Campinas
Semestre: 1º semestre de 2026
Disciplina: MC714 - Sistemas Distribuídos
Projeto: node1, node2, node3 e resource

Documento preparado em português claro, com foco no que foi implementado, corrigido e
validado localmente.

Sumário

1	Introdução	2
2	Objetivos e escopo	2
3	Arquitetura do sistema	3
4	Desenvolvimento e implementação	4
4.1	Base do projeto	4
4.2	Modelo de mensagens e estado	4
4.3	Correções de Lamport e do log de eventos	4
4.4	Exclusão mútua distribuída	5
4.5	Eleição de líder	5
4.6	Ferramentas e dependências	6
4.7	Marcos executados	6
5	Validação, testes e experimentos	6
5.1	Resultados locais validados	7
5.2	Documentação de apoio	7
6	Conclusão	8
7	Referências	8

1 Introdução

Este relatório registra, de forma objetiva, o que já foi construído e o que foi corrigido na revisão local do Trabalho 2 da MC714. O projeto foi estruturado como uma implementação acadêmica de sistemas distribuídos com comunicação HTTP/JSON real entre três nós e um observador externo.

A revisão recente teve dois focos complementares. O primeiro foi técnico: alinhar o código com a semântica esperada para o relógio de Lamport, a exclusão mútua de Ricart-Agrawala e a eleição de líder com Bully. O segundo foi documental: aumentar a rastreabilidade entre teoria, código, testes, smoke tests, experimentos e resultados versionados.

O resultado é uma base local mais clara para avaliação acadêmica, ainda sem extrapolar o escopo original do trabalho.

2 Objetivos e escopo

O objetivo geral foi manter um sistema distribuído simples, mas suficientemente completo para demonstrar comportamento real entre processos independentes executando em contêineres separados.

Os objetivos específicos foram:

- executar três nós idênticos em processos distintos;
- trocar mensagens entre os nós usando HTTP/JSON;
- manter estado local em memória, sem banco de dados ou arquivo compartilhado;
- implementar e validar o relógio lógico de Lamport;
- implementar e validar a exclusão mútua distribuída com Ricart-Agrawala;
- implementar e validar a eleição de líder com Bully;
- registrar a seção crítica em um observador externo chamado **resource**;
- criar testes unitários, smoke tests e experimentos reproduzíveis;
- documentar decisões, limitações e rastreabilidade entre teoria e código.

O escopo permaneceu intencionalmente restrito. O projeto não inclui consenso, replicação, persistência, autenticação, frontend, dashboard, Kubernetes, banco de dados ou tolerância a partições. Essas escolhas mantiveram o trabalho alinhado ao enunciado e preservaram a comparabilidade com a teoria utilizada.

3 Arquitetura do sistema

O sistema foi organizado em quatro serviços principais sob Docker Compose. Os três nós compartilham o mesmo código-fonte e mudam apenas por configuração de ambiente.

Componente	Papel	Observações
node1, node2, node3	Nós distribuídos	Executam FastAPI, Uvicorn e HTTPX; cada um mantém estado local em memória.
resource	Observador	Registra entrada, saída e sobreposição na seção crítica; não decide prioridade.
docker-compose.yml	Orquestração	Sobe a pilha local com um contêiner por nó.
Dockerfile	Empacotamento	Define a imagem base da aplicação.
scripts/	Automação	Contém smoke tests, demonstração e experimentos.
tests/	Validação	Suíte unitária para relógio, mutex, eleição e estado.

Tabela 1: Componentes principais do projeto

A comunicação entre os processos usa envelopes JSON enviados por HTTP. Essa decisão preserva a visibilidade do fluxo de mensagens e facilita a inspeção por logs e por scripts de validação. Cada envio aparece como chamada HTTP real entre contêineres, o que torna o comportamento observável de ponta a ponta.

Os endpoints principais dos nós incluem:

- GET /health para verificação básica;
- GET /state para inspecionar o estado local;
- GET /events para recuperar eventos registrados;
- POST /messages para receber mensagens entre nós;
- POST /commands/local-event para gerar evento local;
- POST /commands/send-message para enviar mensagem a outro nó;
- POST /commands/request-critical-section para solicitar a seção crítica;
- POST /commands/start-election para disparar uma eleição manual.

4 Desenvolvimento e implementação

4.1 Base do projeto

A primeira etapa foi estruturar o ambiente executável e reproduzível. O repositório passou a usar Python 3.12, FastAPI, Uvicorn e HTTPX, com organização em `src/`, `tests/`, `scripts/` e `docs/`. O empacotamento foi definido em `pyproject.toml`, e a execução local foi padronizada com Docker Compose.

Essa base permitiu separar com clareza o que é configuração, o que é implementação e o que é automação. Também facilitou a evolução incremental do projeto sem perder rastreabilidade.

4.2 Modelo de mensagens e estado

O módulo `src/node/models.py` define os tipos de mensagens e comandos usados pelo sistema. O envelope comum inclui identificador do remetente, identificador da mensagem, tempo lógico e um `payload` flexível. Isso permitiu reusar a mesma infraestrutura para mensagens de aplicação, mutex e eleição.

O estado de cada nó fica concentrado em memória e é gerenciado por uma classe central em `src/node/state.py`. Ali estão as responsabilidades de:

- registrar eventos locais e recebidos;
- atualizar o relógio de Lamport;
- controlar estados da exclusão mútua;
- administrar o estado de eleição e líder;
- expor snapshots para inspeção via API.

4.3 Correções de Lamport e do log de eventos

A revisão recente formalizou a diferença entre evento Lamport e anotação de observabilidade. Os registros que avançam ou atualizam o relógio foram classificados como `lamport_event`; os registros que apenas detalham o mesmo acontecimento foram classificados como `annotation`.

Também ficou explícito que o timestamp da mensagem é o timestamp do evento de envio e que o recebimento atualiza o relógio antes do registro do evento correspondente. Essa separação evita interpretar o log de `/events` como uma sequência um-para-um de eventos teóricos.

4.4 Exclusão mútua distribuída

A exclusão mútua foi modelada com o algoritmo de Ricart-Agrawala. Foram adotados os estados `RELEASED`, `WANTED` e `HELD`, e a prioridade segue o par (`request_timestamp`, `node_id`).

Durante uma solicitação de seção crítica, o nó:

1. fixa um `request_timestamp` para toda a tentativa;
2. envia `MUTEX_REQUEST` para os outros nós;
3. aguarda `MUTEX_REPLY` até receber todas as respostas esperadas;
4. entra na seção crítica;
5. notifica o observador `resource` na entrada e na saída;
6. libera respostas adiadas ao final.

A revisão também renomeou o timeout que encerra uma tentativa sem sucesso para `MUTEX_ABORTED`. Isso separa saída normal, timeout e erro de transporte, evitando ambiguidades no log e nos testes.

O observador `resource` foi propositalmente mantido fora da decisão de acesso. Ele só registra `ENTER` e `EXIT` e conta sobreposições, o que ajuda a verificar se a exclusão mútua está funcionando sem centralizar o controle.

4.5 Eleição de líder

A eleição de líder foi implementada com uma adaptação didática do Bully. Os nós trocam mensagens `ELECTION`, `ELECTION_OK`, `COORDINATOR` e `HEARTBEAT`. O maior `node_id` ativo deve assumir como líder.

A revisão recente corrigiu a aceitação de coordenador e heartbeat para respeitar três critérios ao mesmo tempo:

- o identificador do próprio nó;
- o líder atualmente conhecido;
- a prioridade anunciada na mensagem.

Com isso, um líder inferior não substitui um líder superior e um anúncio coerente de nó inferior pode provocar nova eleição quando apropriado. O identificador `election_id` continua sendo uma adaptação local para ignorar respostas antigas ou duplicadas.

4.6 Ferramentas e dependências

A revisão também tornou `pydantic` uma dependência direta do projeto. Isso evita depender apenas da instalação transitiva trazida por FastAPI e deixa a configuração mais explícita.

Outro ponto de rastreabilidade foi a política de resultados. Os artefatos em `docs/` `results/` passaram a ser versionáveis. Essa decisão alinha o README com os dados efetivamente produzidos.

4.7 Marcos executados

Em termos práticos, o trabalho passou pelos seguintes marcos:

- configuração do runtime e da orquestração local;
- implementação dos nós distribuídos com API HTTP;
- adição do relógio de Lamport;
- adição da exclusão mútua com observador externo;
- adição da eleição de líder e do heartbeat;
- revisão da aceitação de coordenador e heartbeat;
- classificação explícita entre evento e anotação;
- criação dos smoke tests e da demonstração automatizada;
- criação dos experimentos e captura dos resultados;
- fechamento com documentação de decisão, rastreabilidade e referências.

5 Validação, testes e experimentos

A validação foi organizada em três camadas. A primeira camada é a suíte de testes unitários em `tests/unit/`, cobrindo relógio, mutex, eleição, modelos e estado. A segunda camada é a bateria de smoke tests em `scripts/`, usada para exercitar o sistema real em execução distribuída. A terceira camada é a coleta de experimentos reproduzíveis, com saída estruturada para análise posterior.

Os scripts principais são:

- `scripts/smoke_http.py`;
- `scripts/smoke_lamport.py`;

- `scripts/smoke_mutex.py`;
- `scripts/smoke_election.py`;
- `scripts/demo.py`;
- `scripts/run_experiments.py`.

5.1 Resultados locais validados

Na revisão local, a suíte de testes executada com a `virtualenv` passou com 62 testes. A checagem de importação também confirmou o uso direto de `pydantic` no ambiente do projeto.

Ao mesmo tempo, o Python do sistema ainda não possuía as dependências do projeto, e o Docker Engine não estava disponível no host no momento da revisão. Por isso, os smoke tests distribuídos e a regeneração dos experimentos não puderam ser executados novamente nesta passagem. Os resultados versionados abaixo permanecem como artefatos históricos do repositório:

- `docs/results/README.md`;
- `docs/results/raw-results.json`;
- `docs/results/experiment-summary.csv`.

5.2 Documentação de apoio

A documentação do projeto foi mantida de forma complementar ao código:

- `README.md` resume a arquitetura, os comandos e os resultados;
- `docs/decision-log.md` registra as decisões de projeto;
- `docs/algorithm-traceability.md` liga teoria, código e testes;
- `docs/references.md` consolida as referências acadêmicas e técnicas.

Isso reduz ambiguidade, facilita auditoria e ajuda a reconstituir o raciocínio usado em cada parte da implementação.

6 Conclusão

O projeto chegou a uma versão funcional e documentada de um sistema distribuído didático. Foram implementados os três pilares esperados do trabalho: comunicação entre nós, exclusão mútua distribuída e eleição de líder. A revisão recente também corrigiu a parte mais sensível para auditoria: prioridade do Bully, classificação dos eventos, rastreabilidade das fontes e política de resultados.

Do ponto de vista técnico, os principais ganhos foram:

- uso real de HTTP/JSON entre contêineres;
- controle de estado local por nó;
- validação por testes unitários e smoke tests;
- coleta de experimentos com rastreabilidade;
- documentação das decisões de projeto.

Como limite assumido desde o início, o trabalho não tenta resolver consenso, particionamento de rede ou persistência durável. Esses itens ficaram fora do escopo para preservar foco e coerência com o enunciado.

Em resumo, o que já foi feito até aqui forma uma base sólida para demonstrar conceitos clássicos de sistemas distribuídos com execução local reproduzível e clareza suficiente para avaliação acadêmica.

7 Referências

Referências

- [1] LAMPORT, Leslie. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, v. 21, n. 7, p. 558–565, 1978. DOI: <https://doi.org/10.1145/359545.359563>.
- [2] RICART, Glenn; AGRAWALA, Ashok K. *An Optimal Algorithm for Mutual Exclusion in Computer Networks*. Communications of the ACM, v. 24, n. 1, p. 9–17, 1981. DOI: <https://doi.org/10.1145/358527.358537>.
- [3] GARCIA-MOLINA, Hector. *Elections in a Distributed Computing System*. IEEE Transactions on Computers, v. C-31, n. 1, p. 48–59, 1982. DOI: <https://doi.org/10.1109/TC.1982.1675885>.
- [4] FASTAPI. Documentação oficial. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 5 jul. 2026.

- [5] HTTPX. Documentação oficial. Disponível em: <https://www.python-httpx.org/>. Acesso em: 5 jul. 2026.
- [6] UVICORN. Documentação oficial. Disponível em: <https://www.uvicorn.org/>. Acesso em: 5 jul. 2026.
- [7] PYTHON SOFTWARE FOUNDATION. *asyncio synchronization primitives* e *time.monotonic*. Disponível em: <https://docs.python.org/3/library/asyncio-sync.html> e <https://docs.python.org/3/library/time.html#time.monotonic>. Acesso em: 5 jul. 2026.